Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

Implementación y análisis del Generador Congruencial Lineal en Python y Java

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Emilio Josue Flores Gallardo
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	005
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	José O. Guamán Q.
Fecha	29 y 30 de octubre de 2026
Horario	Jueves 10h30 a 11h30 / Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas
Repositorio Git	Repositorio GitHub

2. Objetivo(s) de la Práctica

Objetivo general: comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método LCG.

Objetivos específicos:

- Implementar un generador LCG en Python.
- Implementar el mismo algoritmo en Java.
- Comparar las secuencias generadas y analizar sus propiedades.

3. Materiales, Reactivos, Equipos y Herramientas

Software y herramientas:

- **Python 3.12+ (con `Matplotlib`):** usado para programar la primera implementación del generador LCG, el verificador del teorema de Hull–Dobell y los gráficos de secuencia e histogramas.

- **Java SE / JDK (con `Swing` y `AWT`):** usado para programar la segunda implementación equivalente con aritmética de 64 bits (`long`) y renderizado gráfico nativo en ventana.
- **Visual Studio Code y terminal GNU/Linux:** entorno de desarrollo y ejecución de ambos proyectos.
- **Git y GitHub:** control de versiones y almacenamiento del repositorio.

Equipos: computador con procesador de 2 núcleos o superior, 4 GB de RAM y sistema operativo GNU/Linux.

4. Procedimiento / Metodología Ejecutada

4.1. Fundamento teórico

El **Generador Congruencial Lineal** (LCG, *Linear Congruential Generator*) produce una secuencia determinista de números enteros pseudoaleatorios a partir de la relación recursiva:

$$X_{n+1} = (aX_n + c) \bmod m \tag{1}$$

donde X_0 es la semilla inicial, a es el multiplicador, c es el incremento y m es el módulo ($m > 0$). Para transformar los enteros generados en variables uniformes dentro del intervalo $[0, 1)$, cada valor se normaliza dividiéndolo entre el módulo:

$$U_n = \frac{X_n}{m} \tag{2}$$

De acuerdo con el **Teorema de Hull–Dobell**, un generador LCG mixto ($c > 0$) alcanza su período máximo posible ($P = m$) para cualquier semilla inicial si y solo si cumple tres condiciones:

1. c y m son coprimos, es decir, $\gcd(c, m) = 1$.
2. $(a - 1)$ es divisible por todo factor primo p que divida a m.
3. Si m es múltiplo de 4, entonces $(a - 1)$ también debe ser múltiplo de 4.

En la Tabla 1 se detallan los parámetros establecidos en la guía para ambos lenguajes.

Tabla 1: Parámetros de configuración del generador LCG en Python y Java.

Parámetro	Valor	Función en el generador
Multiplicador (a)	1664525	Factor de escala lineal; cumple $(a - 1) = 1664524$, divisible por 2 y por 4.
Incremento (c)	1013904223	Constante aditiva impar; cumple $\gcd(c, 2^{32}) = 1$.
Módulo (m)	$2^{32} = 4294967296$	Cota superior del estado entero y período teórico máximo del ciclo.
Semilla base (X_0)	12345	Valor inicial para el Escenario Base exigido por la guía.
Semilla alternativa (X_0')	98765	Valor inicial para evaluar la sensibilidad del generador ante el cambio de semilla.
Tamaño de muestra (N)	10000	Cantidad total de números pseudoaleatorios generados por escenario.
Muestra en consola	20	Primeras observaciones impresas y comparadas entre escenarios.

4.2. Las 7 etapas de un proyecto de simulación

Para el desarrollo de la práctica se siguieron las siete etapas de un proyecto de simulación, aplicadas a la implementación y validación cruzada del generador LCG en Python y Java:

- **Etapa 1. Definición del problema**
  Se planteó la necesidad de implementar un mismo algoritmo Generador Congruencial Lineal (LCG) en Python y Java para generar $10\,000$ valores, comprobar que ambos lenguajes entreguen exactamente la misma secuencia y analizar el impacto de cambiar la semilla inicial.
- **Etapa 2. Formulación conceptual**
  Se definió el modelo recursivo $X_{n+1} = (aX_n + c) \bmod m$ junto con su normalización $U_n = X_n/m$, el criterio de detección de ciclos empíricos mediante tablas hash y las tres reglas matemáticas del teorema de Hull–Dobell para verificar el período completo.
- **Etapa 3. Recolección y análisis de datos**
  Se fijaron los parámetros de la guía: $a = 1664525$, $c = 1013904223$, $m = 2^{32}$ y $N = 10\,000$, estableciendo dos escenarios de prueba: Escenario Base con semilla $X_0 = 12345$ y Escenario Semilla Alternativa con $X_0 = 98765$.
- **Etapa 4. Construcción del modelo**
  Se desarrollaron dos proyectos equivalentes en Python y Java bajo una arquitectura limpia MVC combinada con capas de generación (`generator`) y servicios de análisis y graficación (`service`), utilizando enteros de 64 bits (`long`) en Java para evitar el desbordamiento de 32 bits al multiplicar $a \cdot X_n$.
- **Etapa 5. Verificación y validación**
  Se contrastaron las primeras 20 observaciones entre Python y Java comprobando que producen exactamente los mismos valores enteros X_n y normalizados U_n hasta el duodécimo decimal, además de verificar que los $10\,000$ datos generados son únicos y cumplen Hull–Dobell ($P = 2^{32}$).
- **Etapa 6. Experimentación y análisis de resultados**
  Se ejecutaron ambos escenarios de semilla, comparando la diferencia absoluta $|U_1(n) - U_2(n)|$ en las primeras 20 iteraciones y graficando tanto el comportamiento de los primeros 200 valores como los histogramas de 20 intervalos sobre los $10\,000$ datos.
- **Etapa 7. Implementación y documentación**
  Se organizaron ambos códigos fuente dentro de las carpetas `Python/` y `Java/` en el repositorio de GitHub junto con su archivo `.gitignore`, documentando las tablas comparativas, capturas divididas y conclusiones en el presente informe.

4.3. Arquitectura MVC con capa de Servicios en Python y Java

Se diseñó la misma arquitectura modular en ambos lenguajes separando el proyecto en cinco paquetes: `model`, `generator`, `service`, `view` y `controller`, aplicando normas de *Clean Code* y principios **SOLID** (Tabla 2):

Tabla 2: Estructura modular simétrica en Python y Java (MVC + Generator/Service).

Módulo (Python / Java)	**Responsabilidad dentro de la arquitectura**
`model/model.py` `model/Model.java`	Define las clases inmutables de datos: `LCGScenario`, `PeriodAnalysisResult` y `SimulationRun`.
`generator/number_generator.py` `generator/NumberGenerator.java`	Calcula las secuencias $X_n = (aX_{n-1} + c) \bmod m$ y $U_n = X_n/m$ para los $N = 10\,000$ pasos.
`service/analyzer.py` `service/Analyzer.java`	Busca repeticiones con tablas hash en $O(N)$ y verifica el teorema de Hull–Dobell mediante $\gcd$ y factores primos.
`service/graphic_generator.py` `service/GraphicGenerator.java`	Dibuja la gráfica de los primeros 200 valores U_n y el histograma de 20 intervalos con la cota esperada.
`view/view.py` `view/View.java`	Imprime en consola las primeras 20 observaciones, el análisis de período y la tabla comparativa de semillas.
`controller/controller.py` `controller/Controller.java`	Orquesta la ejecución de los escenarios y entrega los resultados a la vista de consola y al graficador.
`main.py` `Main.java`	Punto de entrada que configura las semillas `12345` y `98765` e inicia el controlador.

5. Resultados

5.1. Comparación numérica entre Python y Java y cambio de semilla

Al ejecutar ambos programas con los mismos parámetros ($a = 1664525$, $c = 1013904223$, $m = 2^{32}$), tanto Python como Java producen exactamente la misma secuencia de enteros X_n y valores normalizados U_n. En la Tabla 3 se muestran las primeras 10 observaciones verificadas en ambos lenguajes para la semilla base ($X_0 = 12345$) frente a la semilla alternativa ($X_0' = 98765$), junto con su diferencia absoluta $|U_1(n) - U_2(n)|$.

Tabla 3: Comparación de observaciones generadas en Python y Java (Semilla 1 = 12345 vs. Semilla 2 = 98765).

n	$X_1(n)$ **(Py = Java)**	$U_1(n)$ **(Sem. 12345)**	$X_2(n)$ **(Py = Java)**	$U_2(n)$ **(Sem. 98765)**	**Diferencia** $\lvert U_1 - U_2 \rvert$
1	87628868	0.0204026858	2201958600	0.5126834399	0.4922807541
2	71072467	0.0165478482	2731312519	0.6359332528	0.6193854046
3	2332838374	0.5431562597	135349706	0.0315135594	0.5116427003
4	1761074861	0.4100321936	1473772193	0.3431393276	0.0668928660
5	313684584	0.0730353837	972805004	0.2264988162	0.1534634325
6	1852882087	0.4314077276	3148966059	0.7331757953	0.3017680677
7	2690497306	0.6264302192	1298940174	0.3024330764	0.3239971428
8	1882405609	0.4382817096	273755805	0.0637387402	0.3745429694
9	1456332196	0.3390787626	2991532632	0.6965204673	0.3574417047
10	4226455299	0.9840483076	4240292791	0.9872701161	0.0032218085

Asimismo, en la Tabla 4 se resume el análisis de período arrojado por el módulo `Analyzer` en ambos lenguajes para los $10\,000$ valores de cada escenario.

Tabla 4: Resultados del análisis de período empírico y teórico (Hull–Dobell) en Python y Java.

Métrica evaluada	**Escenario Base (12345)**	**Semilla Alternativa (98765)**
Tamaño de muestra evaluada (N)	$10\,000$ valores	$10\,000$ valores
Valores únicos en la muestra	$10\,000$ ($100\,\%$)	$10\,000$ ($100\,\%$)
Ciclo empírico detectado en muestra	No se repiten valores	No se repiten valores
Cumple Teorema de Hull–Dobell	Sí ($\gcd = 1$, $p = 2$, div 4)	Sí ($\gcd = 1$, $p = 2$, div 4)
Período teórico exacto (P)	$m = 4\,294\,967\,296$ (2^{32})	$m = 4\,294\,967\,296$ (2^{32})

5.2. Salidas en consola divididas (Python y Java)

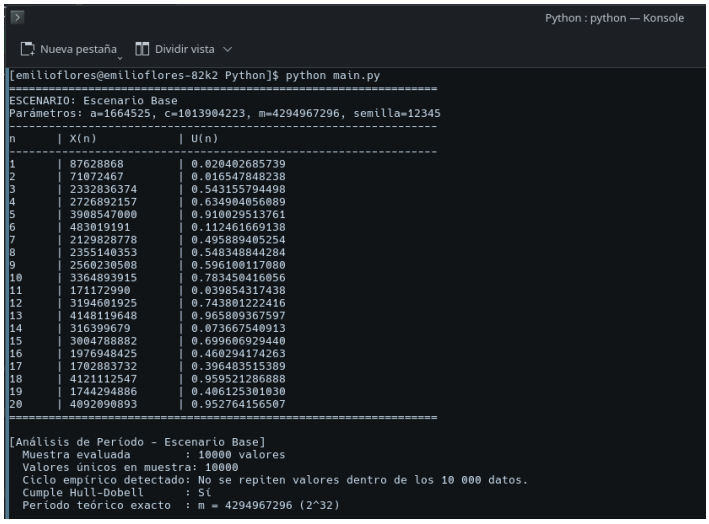


Figura 1: Salida en consola en Python (Parte 1/3)

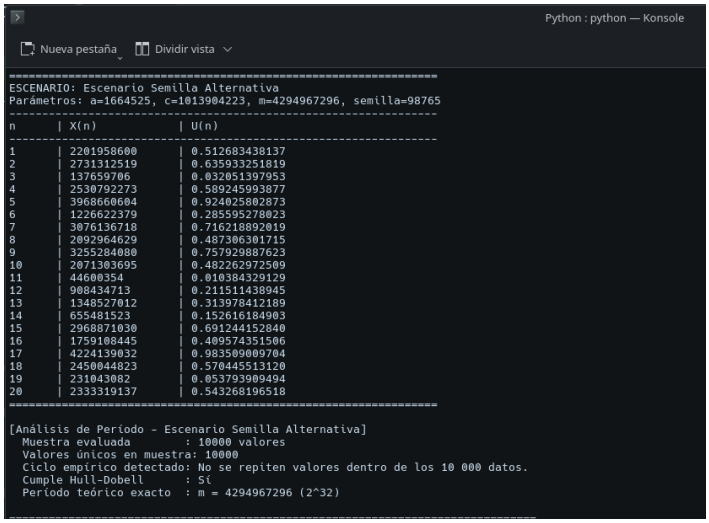


Figura 2: Salida en consola en Python (Parte 2/3)

```
================================================================================
COMPARACIÓN DE LAS PRIMERAS 20 OBSERVACIONES (Semilla 1 = 12345 vs Semilla 2 = 98765)
--------------------------------------------------------------------------------
n     | X1(n)         | U1(n)          || X2(n)         | U2(n)          | |U1 - U2|
--------------------------------------------------------------------------------
1     | 87628868      | 0.0204026857   || 2201958600     | 0.5126834381   | 0.4922807524
2     | 71072467      | 0.0165478482   || 2731312519     | 0.6359332518   | 0.6193854036
3     | 2332836374    | 0.5431557945   || 137659706      | 0.0320513980   | 0.5111043965
4     | 2726892157    | 0.6349040561   || 2530792273     | 0.5892459939   | 0.0456580622
5     | 3908547000    | 0.9100295138   || 3968660604     | 0.9240258029   | 0.0139962891
6     | 483019191     | 0.1124616691   || 1226622379     | 0.2855952780   | 0.1731336089
7     | 2129828778    | 0.4958894053   || 3076136718     | 0.7162188920   | 0.2203294868
8     | 2355140353    | 0.5483488443   || 2092964629     | 0.4873063017   | 0.0610425426
9     | 2560230508    | 0.5961001171   || 3255284080     | 0.7579298876   | 0.1618297705
10    | 3364893915    | 0.7834504161   || 2071303695     | 0.4822629725   | 0.3011874435
11    | 171172990     | 0.0398543174   || 44600354       | 0.0103843291   | 0.0294699883
12    | 3194601925    | 0.7438012224   || 908434713      | 0.2115114389   | 0.5322897835
13    | 4148119648    | 0.9658093676   || 1348527012     | 0.3139784122   | 0.6518309554
14    | 316399679     | 0.0736675409   || 655481523      | 0.1526161849   | 0.0789486440
15    | 3004788882    | 0.6996069294   || 2968871030     | 0.6912441528   | 0.0083627766
16    | 1976948425    | 0.4602941743   || 1759108445     | 0.4095743515   | 0.0507198228
17    | 1702883732    | 0.3964835154   || 4224139032     | 0.9835090097   | 0.5870254943
18    | 4121112547    | 0.9595212869   || 2450044823     | 0.5704455131   | 0.3890757738
19    | 1744294886    | 0.4061253010   || 231043082      | 0.0537939095   | 0.3523313915
20    | 4092090893    | 0.9527641565   || 2333319137     | 0.5432681965   | 0.4094959600
================================================================================
```

Figura 3: Salida en consola en Python (Parte 3/3)

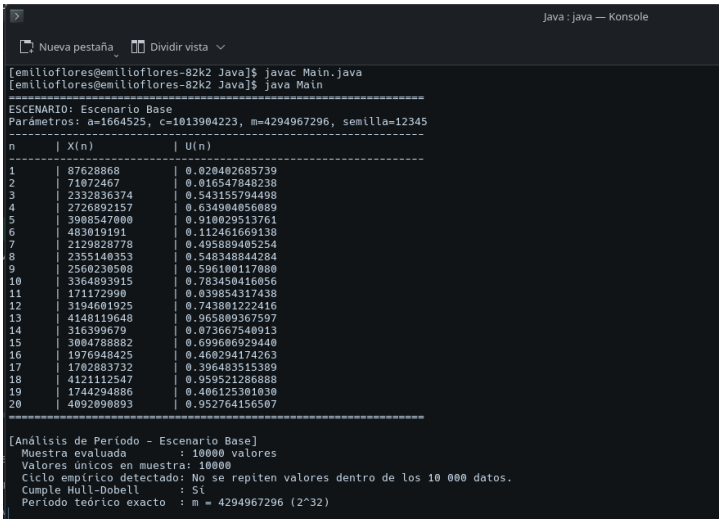


Figura 4: Salida en consola en Java (Parte 1/3)

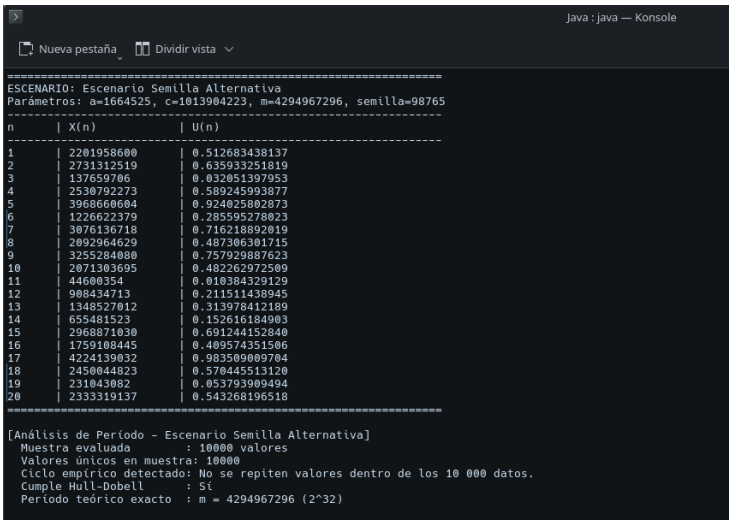


Figura 5: Salida en consola en Java (Parte 2/3)

```
================================================================================
COMPARACIÓN DE LAS PRIMERAS 20 OBSERVACIONES (Semilla 1 = 12345 vs Semilla 2 = 98765)
--------------------------------------------------------------------------------
n     | X1(n)         | U1(n)          || X2(n)         | U2(n)          | |U1 - U2|
--------------------------------------------------------------------------------
1     | 87628868      | 0.0204026857   || 2201958600   | 0.5126834381   | 0.4922807524
2     | 71072467      | 0.0165478482   || 2731312519   | 0.6359332518   | 0.6193854036
3     | 2332836374    | 0.5431557945   || 137659706    | 0.0320513980   | 0.5111043965
4     | 2726892157    | 0.6349040561   || 2530792273   | 0.5892459939   | 0.0456580622
5     | 3908547000    | 0.9100295138   || 3968660604   | 0.9240258029   | 0.0139962891
6     | 483019191     | 0.1124616691   || 1226622379   | 0.2855952780   | 0.1731336089
7     | 2129828778    | 0.4958894053   || 3076136718   | 0.7162188920   | 0.2203294868
8     | 2355140353    | 0.5483488443   || 2092964629   | 0.4873063017   | 0.0610425426
9     | 2560230508    | 0.5961001171   || 3255284080   | 0.7579298876   | 0.1618297705
10    | 3364893915    | 0.7834504161   || 2071303695   | 0.4822629725   | 0.3011874435
11    | 171172990     | 0.0398543174   || 44600354     | 0.0103843291   | 0.0294699883
12    | 3194601925    | 0.7438012224   || 908434713    | 0.2115114389   | 0.5322897835
13    | 4148119648    | 0.9658093676   || 1348527012   | 0.3139784122   | 0.6518309554
14    | 316399679     | 0.0736675409   || 655481523    | 0.1526161849   | 0.0789486440
15    | 3004788882    | 0.6996069294   || 2968871030   | 0.6912441528   | 0.0083627766
16    | 1976948425    | 0.4602941743   || 1759108445   | 0.4095743515   | 0.0507198228
17    | 1702883732    | 0.3964835154   || 4224139032   | 0.9835090097   | 0.5870254943
18    | 4121112547    | 0.9595212869   || 2450044823   | 0.5704455131   | 0.3890757738
19    | 1744294886    | 0.4061253010   || 231043082    | 0.0537939095   | 0.3523313915
20    | 4092090893    | 0.9527641565   || 2333319137   | 0.5432681965   | 0.4094959600
================================================================================
```

Figura 6: Salida en consola en Java (Parte 3/3)

5.3. Gráficas de secuencia e histogramas divididas (Python y Java)

Las ventanas gráficas generadas por `Matplotlib` (Python) y `Swing/Graphics2D` (Java) también se presentan divididas en dos mitades: la primera mitad corresponde al Escenario Base (`semilla = 12345`) y la segunda mitad al Escenario Semilla Alternativa (`semilla = 98765`).

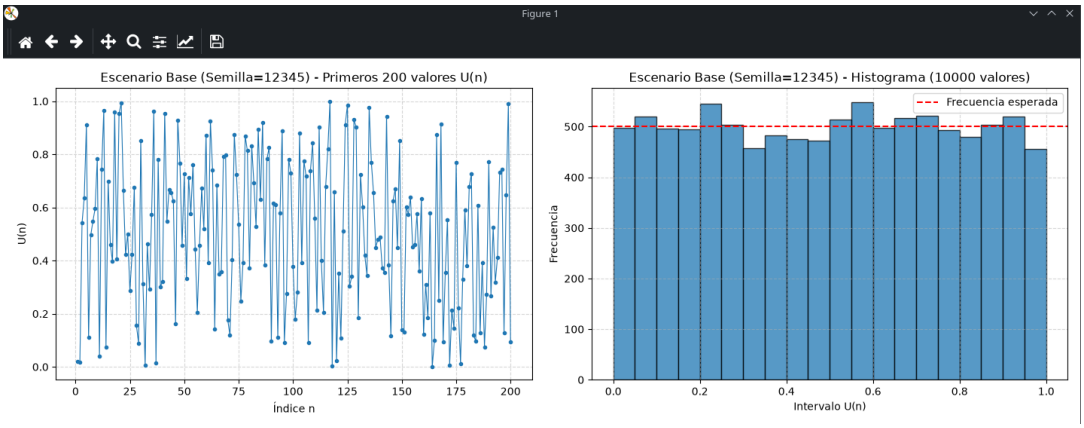


Figura 7: Gráfica en Python (Mitad 1/2 - Semilla 12345): primeros 200 valores U_n e histograma de $10\,000$ datos.

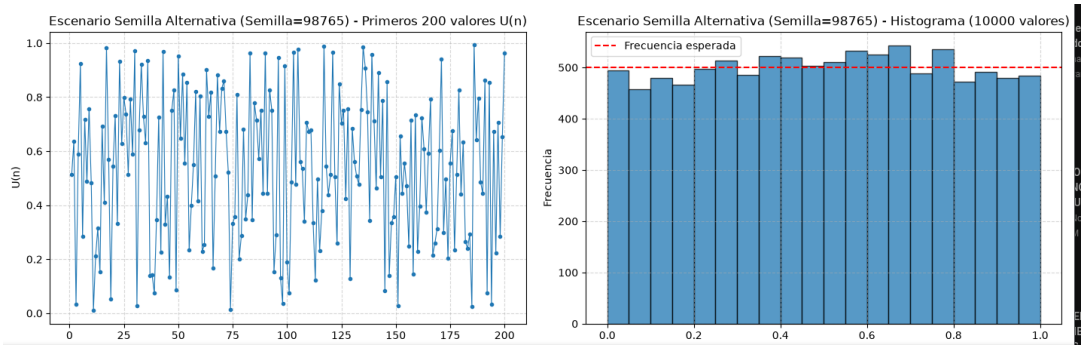


Figura 8: Gráfica en Python (Mitad 2/2 - Semilla 98765): primeros 200 valores U_n e histograma de $10\,000$ datos.

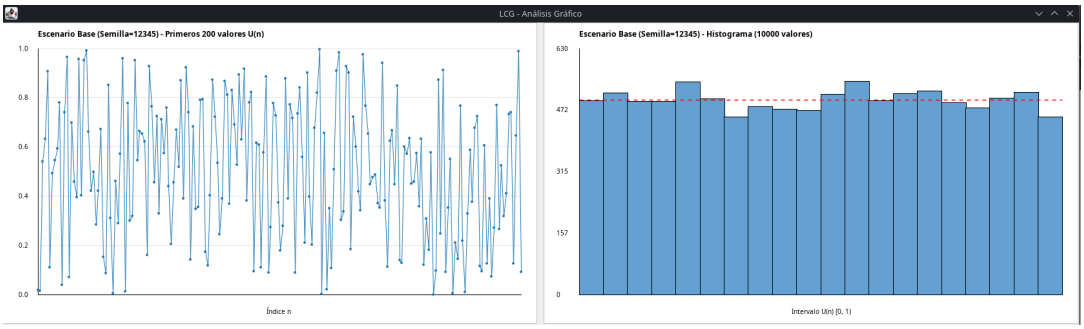


Figura 9: Gráfica en Java Swing (Mitad 1/2 - Semilla 12345): primeros 200 valores U_n e histograma de $10\,000$ datos.

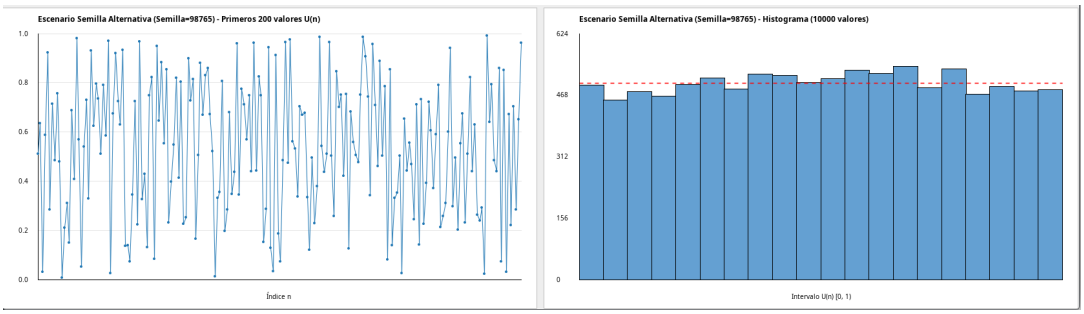


Figura 10: Gráfica en Java Swing (Mitad 2/2 - Semilla 98765): primeros 200 valores U_n e histograma de $10\,000$ datos.

5.4. Análisis de resultados

- **Identidad exacta entre Python y Java:** Como se comprueba en la Tabla 3 y en las capturas de consola, ambos programas entregan exactamente los mismos números enteros ($X_1 = 87628868$, $X_2 = 71072467$, $X_3 = 2332838374$, etc.) y los mismos decimales en U_n. Esto confirma que el algoritmo es determinista y portable cuando en Java se opera con variables `long` de 64 bits para evitar el desbordamiento de enteros de 32 bits.
- **Uniformidad y ausencia de patrones en las gráficas:** En las gráficas de línea de los primeros 200 valores (izquierda de cada figura), los puntos U_n oscilan por todo el rango $[0, 1)$ sin mostrar tendencias crecientes ni ciclos cortos. En los histogramas de 20 clases (derecha de cada figura), las barras de ambos escenarios se mantienen muy cerca de la línea roja punteada de frecuencia teórica esperada ($10\,000/20 = 500$ datos por intervalo), confirmando una distribución empírica uniforme.
- **Efecto del cambio de semilla:** Al cambiar la semilla de 12345 a 98765 manteniendo fijos a, c y m, la trayectoria puntual cambia desde la primera iteración ($U_1 = 0.0204$ frente a $U_1 = 0.5127$, con diferencia de 0.4923). Sin embargo, como los parámetros cumplen el teorema de Hull–Dobell, ambas semillas recorren el mismo ciclo completo de longitud 2^{32}, solo que empezando desde posiciones distintas del anillo modular.

5.5. Cumplimiento de resultados esperados

Tabla 5: Verificación de los resultados esperados en la práctica APE 005.

Resultado esperado según la guía	Cumplimiento en la práctica
Implementación funcional de LCG en Python con $10\,000$ valores.	Implementado en `Python/` generando $10\,000$ datos para las semillas `12345` y `98765`.
Implementación equivalente del algoritmo LCG en Java.	Implementado en `Java/` con aritmética `long` produciendo valores idénticos a Python.
Comparación de las primeras 20 observaciones.	Impresa por consola en ambos lenguajes y sintetizada en la Tabla 3.
Gráficas de la secuencia y análisis del período.	Secuencias de 200 puntos, histogramas de $10\,000$ datos y validación de Hull–Dobell ($P = 2^{32}$).
Cambio de semilla y comparación de resultados.	Evaluado comparando el Escenario Base (`12345`) contra Semilla Alternativa (`98765`).

6. Preguntas de Control

1. **¿Qué significa pseudoaleatorio?**
   Significa que los números son generados mediante una fórmula matemática determinista (como $X_{n+1} = (aX_n + c) \bmod m$), por lo que no son verdaderamente azarosos, pero estadísticamente aparentan ser independientes y distribuirse de forma uniforme en el intervalo $[0, 1)$ [1, 3].

2. **¿Qué función cumple la semilla?**
   Es el valor inicial (X_0) con el que arranca la recurrencia del generador. Cumple dos funciones clave: si se mantiene la misma semilla se puede reproducir exactamente el mismo experimento de simulación, y si se cambia la semilla se obtiene una secuencia diferente para realizar réplicas independientes [1, 4].

3. **¿Qué significa período?**
   Es la cantidad máxima de valores distintos que produce el generador antes de que se repita un número previo y toda la secuencia empiece a ciclarse exactamente en el mismo orden. En un LCG con módulo m, el período máximo posible es $P = m$ [2, 3].

4. **¿Qué ocurre si se utilizan parámetros inadecuados?**
   Si los parámetros a, c y m no cumplen las condiciones del teorema de Hull–Dobell (por ejemplo, si tienen factores comunes mal elegidos), el período se vuelve muy corto, los números se repiten rápidamente, aparecen correlaciones en serie (los puntos caen sobre pocos hiperplanos) y los histogramas dejan huecos sin cubrir en el rango $[0, 1)$, invalidando la simulación [3, 4].

5. **¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?**
   Permite hacer una verificación cruzada de la implementación: al comprobar que Python y Java producen exactamente los mismos números con los mismos parámetros, se garantiza que no existen errores de redondeo ni desbordamiento de enteros (*integer overflow*) y que el modelo de simulación será totalmente reproducible sin importar en qué plataforma se ejecute [1, 4].

7. Conclusiones

- Se implementó el Generador Congruencial Lineal (LCG) en Python y Java bajo una misma arquitectura modular MVC + Servicios, comprobando que al usar los parámetros $a = 1664525$, $c = 1013904223$, $m = 2^{32}$ y semilla 12345, ambos lenguajes generan

exactamente la misma secuencia de valores enteros X_n y normalizados U_n.

- Mediante el analizador de período y la verificación del teorema de Hull–Dobell se demostró que el generador posee período completo ($P = 2^{32} = 4\,294\,967\,296$), obteniendo $10\,000$ valores únicos sin repeticiones y un histograma uniforme alrededor de la frecuencia esperada de 500 observaciones por clase.
- La comparación entre la semilla base (12345) y la semilla alternativa (98765) evidenció que modificar X_0 desplaza por completo el punto de partida de la secuencia generada desde la primera iteración, manteniendo intactas las propiedades estadísticas de uniformidad y longitud de ciclo.

8. Bibliografía / Referencias

1. J. O. Guamán Q., *Guía de Actividades Práctico-Experimentales Nro. 005: Implementación y análisis del Generador Congruencial Lineal en Python y Java*. Carrera de Computación, FEIRNNR, Universidad Nacional de Loja, 2026.
2. Asignatura Simulación, *Diapositivas de la Semana 5: Generación de Números Pseudoaleatorios y Método Congruencial Lineal*. Carrera de Computación, FEIRNNR - UNL, 2026.
3. T. E. Hull y A. R. Dobell, “Random number generators”, *SIAM Review*, vol. 4, no. 3, pp. 230–254, 1962.
4. A. M. Law, *Simulation Modeling and Analysis*, 5.ª ed. New York, NY, EE. UU.: McGraw-Hill Education, 2014.
5. D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3.ª ed. Reading, MA, EE. UU.: Addison-Wesley, 1997.

9. Anexos

Anexo A. Instalación, compilación y ejecución en Python y Java

Repositorio del proyecto: Repositorio GitHub

```
# 1. Clonar el repositorio
git clone https://github.com/NullDistortion/Repositorio-Simulacion.git

# 2. Ejecutar la implementacion en Python
cd Repositorio-Simulacion/Ape5/Python
python3 -m venv .venv
source .venv/bin/activate
pip install matplotlib
python main.py

# 3. Compilar y ejecutar la implementacion en Java
cd ../Java
javac Main.java
java Main
```

Anexo B. Declaración de uso de inteligencia artificial

Siguiendo los principios de transparencia y honestidad de la asignatura, se declara que durante el desarrollo de esta práctica se utilizaron herramientas de inteligencia artificial generativa como apoyo para estructurar la arquitectura simétrica en Python y Java, implementar la verificación del teorema de Hull–Dobell y construir los paneles gráficos en Matplotlib y Java Swing.

Prompt utilizado para el desarrollo de la arquitectura y diseño del código:

> *"Partiendo del contexto teórico sobre "Generador Congruencial Lineal (LCG)" con parámetros* $a = 1664525$*,* $c = 1013904223$*,* $m = 2^{32}$*,* $N = 10000$ *y semillas* 12345 *y* 98765*, implementa el mismo algoritmo tanto en Python como en Java aplicando una arquitectura Modelo-Vista-Controlador (MVC) extendida con capas de generación (`generator`) y servicios (`service`), respetando normas de Clean Code y principios SOLID. En el modelo (`LCGScenario`, `PeriodAnalysisResult`, `SimulationRun`) usa estructuras de datos inmutables; en `NumberGenerator` calcula* $X(n+1) = (aX(n) + c) \bmod m$ *y* $U(n) = X(n)/m$ *cuidando usar enteros de 64 bits (`long`) en Java para evitar desbordamientos; en `Analyzer` detecta ciclos empíricos con tablas hash en* $O(N)$ *y verifica el teorema de Hull-Dobell mediante* $\gcd$ *y factores primos; en `View` imprime las primeras 20 observaciones, el análisis de período y la comparación entre semillas; y en `GraphicGenerator` grafica los primeros 200 valores y el histograma de 20 intervalos usando Matplotlib en Python y Swing/Graphics2D en Java."*

Declaración de Delegación en Inteligencia Artificial Generativa (Traducción GAIDeT):

Los autores declaran el uso de inteligencia artificial generativa en el proceso de investigación y redacción. De acuerdo con la taxonomía GAIDeT (2026), las siguientes tareas fueron delegadas a herramientas de IAG bajo supervisión humana total:

- Generación de ideas (*Idea generation*)
- Generación de código (*Code generation*)
- Optimización de código (*Code optimization*)
- Automatización de procesos (*Process automation*)
- Creación de algoritmos para análisis de datos (*Creation of algorithms for data analysis*)
- Validación (*Validation*)
- Limpieza de datos (*Data cleaning*)
- Análisis de datos (*Data analysis*)
- Visualización (*Visualization*)

La herramienta de IAG utilizada fue: **Gemini Argon Alto**.
La responsabilidad del manuscrito final recae íntegramente en los autores.
Las herramientas de IAG no figuran como autores y no asumen responsabilidad por los resultados finales.
Declaración presentada por: Emilio Josue Flores Gallardo.